

15강

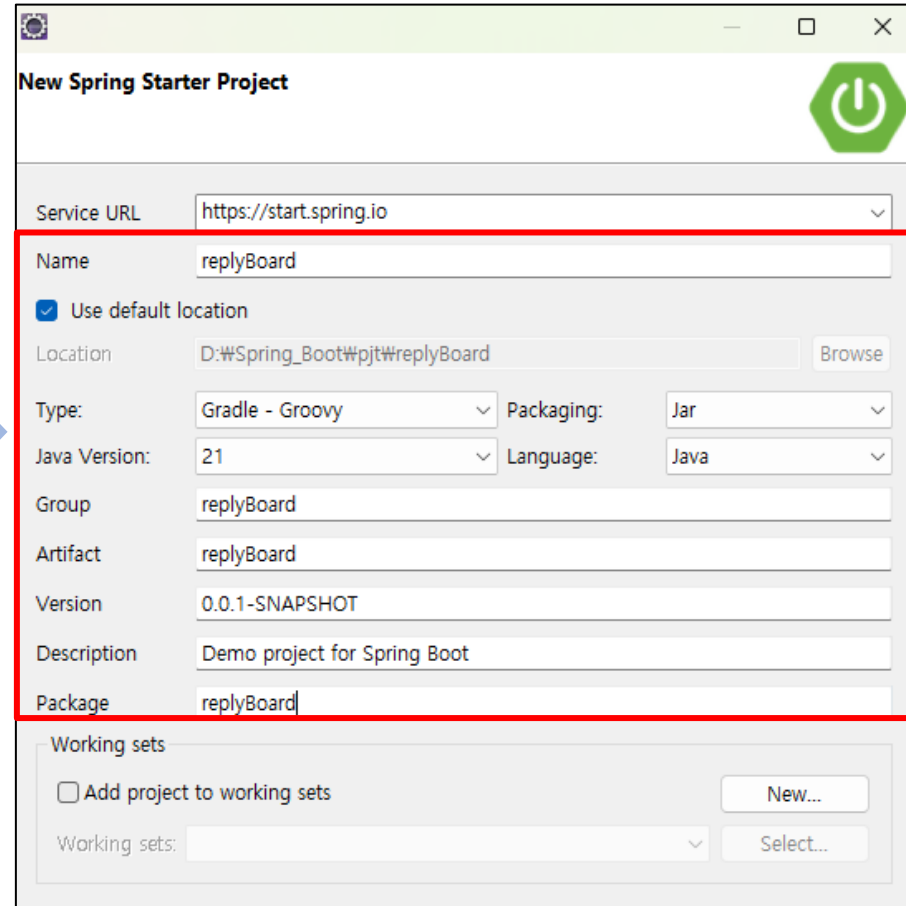
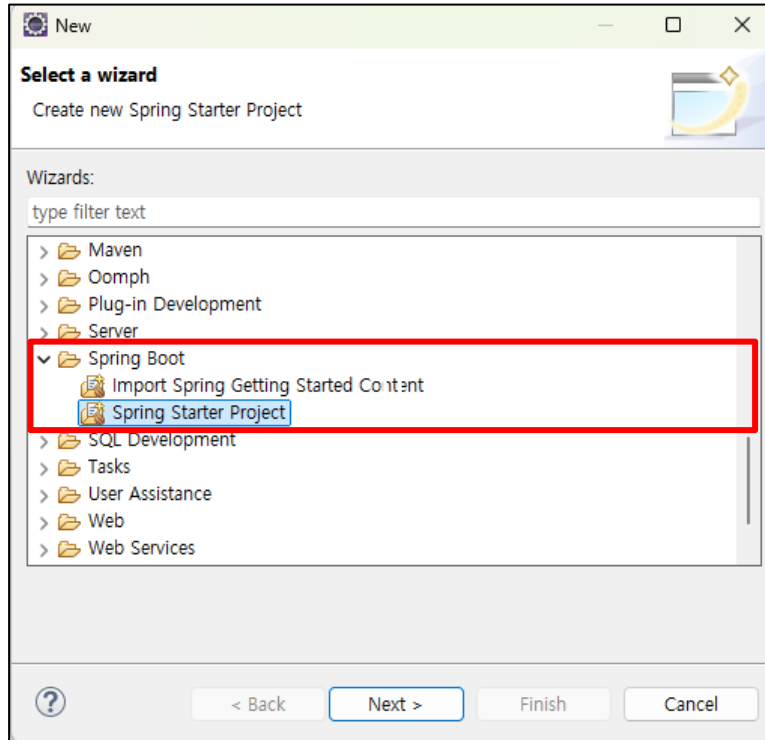
spring MVC MyBatis

답글 게시판(replyboard)



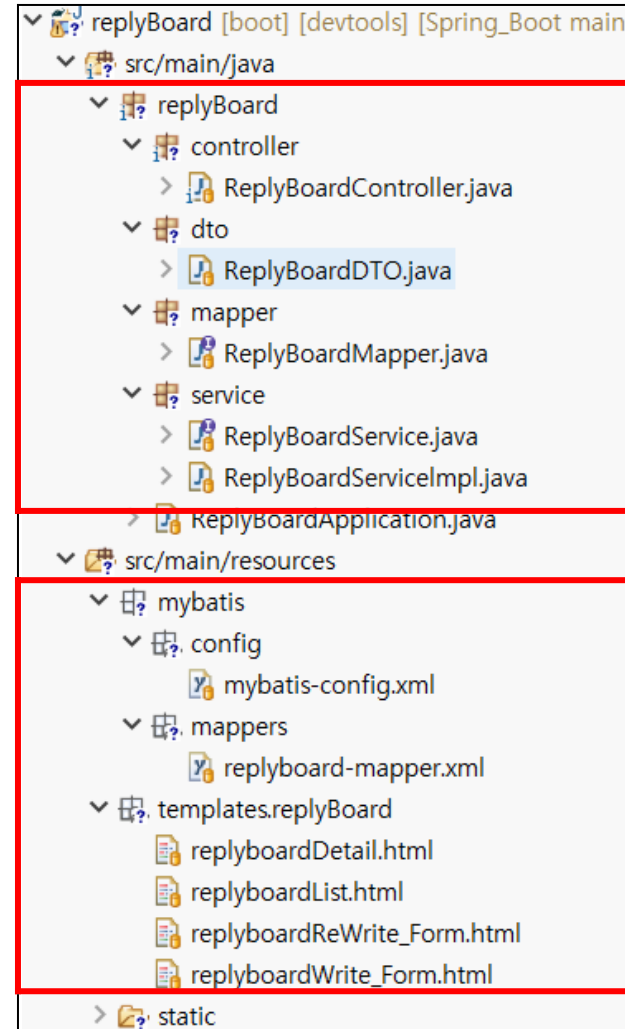
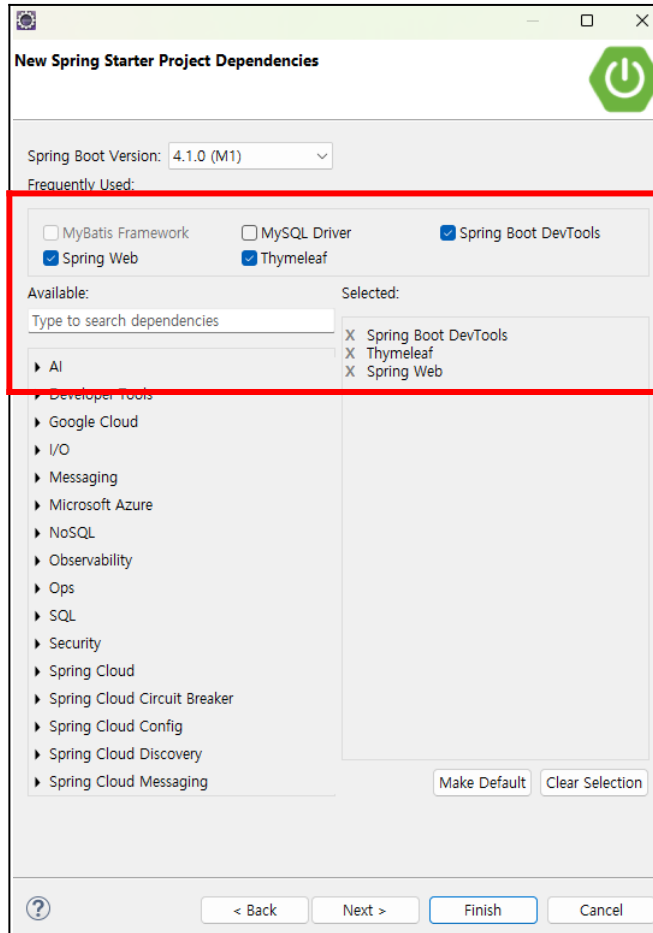


➤ 답글 게시판 프로젝트 생성하기





➤ 답글 게시판 폴더 구조 만들기





➤ 답글 게시판 환경 설정하기

- build.gradle 을 위에서 작성한 프로젝트에서 복사 붙여넣기 한다.
- 반드시 Gradle -> Refresh gradle project 진행한다.

```
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'  
    //implementation 'org.springframework.boot:spring-boot-starter-webmvc'  
    implementation 'org.springframework.boot:spring-boot-starter-web' // * 변경  
    //implementation 'org.springframework.boot:spring-boot-starter-webmvc'  
  
    // Source: https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-jdbc  
    implementation("org.springframework.boot:spring-boot-starter-jdbc")  
  
    // Source: https://mvnrepository.com/artifact/com.mysql/mysql-connector-j  
    implementation("com.mysql:mysql-connector-j")  
  
    // Source: https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-security  
    //implementation("org.springframework.boot:spring-boot-starter-security")  
  
    // Source: https://mvnrepository.com/artifact/org.mybatis.spring.boot/mybatis-spring-boot-starter  
    implementation("org.mybatis.spring.boot:mybatis-spring-boot-starter:4.0.1")  
  
    developmentOnly 'org.springframework.boot:spring-boot-devtools'  
    testImplementation 'org.springframework.boot:spring-boot-starter-thymeleaf-test'  
    testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'  
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'  
}
```



➤ 답글 게시판 환경 설정하기

- application.properties 을 위에서 작성한 프로젝트에서 복사 붙여넣기 한다.
- 반드시 Gradle -> Refresh gradle project 진행한다.

```
spring.application.name=replyBoard
# Tomcat 서버를 내장되어 있다.
# 원래 포트번호는 8080인데 => 8090으로 변경함
# Tomcat server
.port=8090

# Dev Tools spring.devtools.restart.enabled=true

# Thymeleaf #주석 spring.thymeleaf.cache=false
spring.thymeleaf.prefix=classpath:/templates/
spring.thymeleaf.suffix=.html spring.thymeleaf.check-template-location=true

# MySQL
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/springBootDB
spring.datasource.username=root spring.datasource.password=12345678
```

```
# Mybatis
mybatis.config-location=classpath:mybatis/config/mybatis-config.xml
mybatis.mapper-locations=classpath:mybatis/mappers/*.xml

# [추가] MyBatis SQL 로그 출력 설정
# 패키지 경로(com.green.member.mapper)의 로그 레벨을 DEBUG로 설정하면
# 실행되는 SQL문과 전달되는 파라미터(? 값)를 콘솔에서 볼 수 있습니다. logging.level.replyBoard.mapper=DEBUG
```



spring

➤ 게시판 DB 구조

댓글 기능이 존재하는 테이블 구조

컬럼명	설명	비고
Num	글번호	자동증가 Int
Writer	글쓴이	String
Email	이메일	String
Subject	글제목	String
Password	글 비밀번호	String
Reg_date	글 작성날짜	Date
Ref	같은 원글 번호(댓글 묶음 기준)	Int
Re_step	댓글 출력 순서(위 → 아래)	Int
Re_level	댓글 들여쓰기 단계(원글/댓글 구분)	Int
Readcount	조회수	Int
Content	글내용	String



spring

➤ 게시판 DB 구조

글번호	글제목	Ref (글그룹)	Re_step (글스텝)	Re_level (글레벨)
1	봄아~	1	1	1
2	여름아~	2	1	1
3	[re]나도 봄이 좋아	1	2	2

① Ref (그룹 번호): 원본 글과 그에 달린 모든 댓글을 하나의 그룹으로 묶어 주는 역할을 한다.

 설명 멘트: Ref는 “어느 원글에 달린 댓글인가”를 나타낸다.



spring

➤ 게시판 DB 구조

글번호	글제목	Ref (글그룹)	Re_step (글스텝)	Re_level (글레벨)
1	봄아~	1	1	1
2	여름아~	2	1	1
3	[re]나도 봄이 좋아	1	2	2

② Re_step (출력 순서): 같은 Ref 내에서 어떤 순서로 보여줄지 결정한다. 댓글이 추가될 때
마다 기존 댓글들의 순서를 밀어내는 로직을 담당 한다.

🔊 설명 멘트: Re_step은 댓글이 “위에서부터 몇 번째로 보일지”를 정한다.

그래서

[봄아~] (1) 다음에 달린 댓글은 → Re_step = 2



spring

➤ 게시판 DB 구조

글번호	글제목	Ref (글그룹)	Re_step (글스텝)	Re_level (글레벨)
1	봄아~	1	1	1
2	여름아~	2	1	1
3	[re]나도 봄이 좋아	1	2	2

③ Re_level (들여쓰기): 시각적으로 "이것은 답글의 답글이다"라는 것을 보여주는 깊이(Depth) 값이다. UI에서 margin-left 등을 조절하는 용도이다.

원글은 항상 1 닷

글은 부모 글 + 1 대

댓글이면 계속 +1

 설명 멘트: Re_level은 댓글의 "들여쓰기 단계"이다.



spring

➤ 게시판 DB 구조

글번호	글제목	Ref (글그룹)	Re_step (글스텝)	Re_level (글레벨)
1	봄아~	1	1	1
2	여름아~	2	1	1
3	[re]나도 봄이 좋아	1	2	3
4	[re]놀러가자	1	2	2

- **[봄아~]라는 글에 댓글인 [re]놀러가자~를 달면 부모글 그룹을 그대로 쫓아 오기때문에 Ref(글그룹)은 1 이 된다.**
- **Re_step(글스텝)은 [봄아~]가 부모글로 같은 부모글 이기때문에 2가 된다. 스텝은 겹칠 수 있다.**
- **Re_level(글레벨)은 [봄아~]라는 부모글을 제외한 레벨을 모두 1씩 증가시킨다.**



spring

➤ 게시판 DB 구조

- 화면에 출력 할때는 최신글이 반드시 먼저 출력되도록 한다.
- Ref는 내림차순정렬, re_step와 re_level은 오름차순정렬 한다.

글번호	글제목	Ref (글그룹)	Re_step (글스텝)	Re_level (글레벨)
2	여름아	2	1	1
1	봄아~	1	1	1
4	[re]놀러가자	1	2	2
3	[re]나도봄이좋아	1	2	3

ref : 부모글 (원글)

re_step : 화면에 보여주기 위한 위치 값(= 들여쓰기 위한 스텝)

1 : 공백없이 보여줌

2 : 5칸

3 : 10칸

4 : 15칸

re_level : 최신글 부터 보기위한 순서 값

부모글에 속해있는 모든 댓글에 +1 을 한다.



spring

➤ 게시판 DB 구조

글번호	글제목	Ref (글그룹)	Re_step (글스텝)	Re_level (글레벨)
1	베스킨라빈스	1	1	1
2	[re]난 나뚜르	1	2	3
3	[re]나뚜르 나도	1	3	4
4	[re]베스킨 좋아	1	2	2
5				



글번호	글제목	Ref (글그룹)	Re_step (글스텝)	Re_level (글레벨)
1	베스킨라빈스	1	1	1
4	[re]베스킨 좋아	1	2	2
2	[re]난 나뚜르	1	2	3
3	[re]나뚜르 나도	1	3	4



➤ 답글 게시판 - DB 테이블 구조

MySQL -> springbootdb 스키마 -> replyboard 테이블을 생성한다.

```
create table replyBoard(
num int auto_increment primary key,
writer varchar(20),
email varchar(50), subject varchar(50), password varchar(10),
reg_date datetime default now(),
ref int,
re_step int,
re_level int,
readcount int default 0,
content varchar(1000)
);
```

[illegible]



➤ 답글 게시판 – DB 테이블 구조

- src/main/java → replyboard 폴더 → dto → ReplyBoardDTO.java 클래스 파일을 생성 **후 아래처럼 작성한다**
- **반드시** getter, setter **진행한다.**

```
package replyboard.dto;

import java.util.Date;

public class ReplyBoardDTO {
    private int num; // 번호
    private String writer; // 작성자
    private String email; // email
    private String subject; // 제목
    private String password; // 비밀번호
    private Date reg_date; // 작성일
    private int ref; // 같은 원글 번호 (댓글 묶음 기준)
    private int re_step; // 댓글 출력 순서 (위 → 아래)
    private int re_level; // 댓글 들여쓰기 단계 (원글/댓글 구분)
    private int readcount; // 조회수
    private String content; // 글 내용
```

답글 게시판 필요한 Mapper 및 Service





➤ **답글 게시판 작성에 필요한 ReplayMapper 인터페이스의 각 메소드들을 작성한다.**

- src/main/java → replyboard 폴더 → mapper → ReplyBoardMapper.java 파일을 생성한 후 아래 메소드들을 작성한다.

```
package replyboard.mapper;

import java.util.List;

import org.apache.ibatis.annotations.Mapper;

@Mapper
public interface ReplyBoardMapper {

    // 게시글 작성하여 추가하기
    public void insertReplyBoard(ReplyBoardDTO rdto);

    // 게시글 전체 목록 검색
    public List<ReplyBoardDTO> getAllReplyBoard();

    // 하나의 게시글을 리턴받는 메소드 작성
    public ReplyBoardDTO getOneBoard(int num);

    // 답글 작성하여 추가하기
    public void reWriteInsert(ReplyBoardDTO rdto);

    // 답글 작성시 부모글의 re_level 보다 큰 값들을 모두 1씩 증가시키는 메소드
    public void reSqlUpdate(ReplyBoardDTO rdto);

}
```




➤ **답글 게시판 작성에 필요한 Service 인터페이스와 상속받는 클래스를 생성한다.**

- src/main/java → replyboard 폴더 → service → ReplyBoardService.java 인터페이스를 생성한다.
- 인터페이스는 추상메소드만 작성할 수 있음에 유의하자!
- Service를 인터페이스로 작성하는 이유는 실무에서 필요에 따라 구현 클래스를 바꾸어 사용하기 용이하기때문에 자바의 낮은 결합도를 구축하기에 적합하기 때문이다.

```
package replyboard.service;

import java.util.List;

public interface ReplyBoardService {
    // 게시글 작성하여 추가하기
    public void insertReplyBoard(ReplyBoardDTO rdto);

    // 게시글 전체 목록 검색
    public List<ReplyBoardDTO> getAllReplyBoard();

    // 하나의 게시글을 리턴받는 메소드 작성
    public ReplyBoardDTO getOneBoard(int num);

    // 답글 작성하여 추가하기
    public void reWriteInsert(ReplyBoardDTO rdto);

    // 답글 작성시 부모글의 re_level 보다 큰 값들을 모두 1씩 증가시키는 메소드
    public void reSqlUpdate(ReplyBoardDTO rdto);

    public void replyProcess(ReplyBoardDTO rdto);
}
```



➤ 답글 게시판 작성에 필요한 ReplyBoardService 인터페이스를 상속 받는 클래스 생성

- src/main/java → replyboard 폴더 → service → ReplyBoardService.java 인터페이스를 상속 (**implements**) 받는 ReplyBoardServiceImpl.java 파일을 생성한다.
- ReplyBoardServiceImpl.java는 ReplyBoardService 파일을 구현체 이다.

```

1 package replyboard.service;
2
3 import java.util.List;
4
5 @Service
6 public class ReplyBoardServiceImpl implements ReplyBoardService {
7     @Autowired
8     ReplyBoardMapper replyBoardMapper;
9
10    @Override
11    public void insertReplyBoard(ReplyBoardDTO rdto) {
12        replyBoardMapper.insertReplyBoard(rdto);
13    }
14
15    @Override
16    public List<ReplyBoardDTO> getAllReplyBoard() {
17        return replyBoardMapper.getAllReplyBoard();
18    }
19
20    @Override
21    public ReplyBoardDTO getOneBoard(int num) {
22        return replyBoardMapper.getOneBoard(num);
23    }
24

```

```

// reWriteInsert()는 "INSERT만 하는 메소드" 여야 재사용하기 편리하다.
@Override
public void reWriteInsert(ReplyBoardDTO rdto) {
    replyBoardMapper.reWriteInsert(rdto);
}

@Override
public void reSqlUpdate(ReplyBoardDTO rdto) {
    replyBoardMapper.reSqlUpdate(rdto);
}

@Override
public void replyProcess(ReplyBoardDTO rdto) {
    // 반드시 update를 먼저 실행해야 함
    replyBoardMapper.reSqlUpdate(rdto);
    replyBoardMapper.reWriteInsert(rdto);
}

```



➤ 답글 게시판 작성에 필요한 replayboard-mapper.xml 파일 생성

- src/main/resources → templates → mybatis 폴더 → mappers 폴더 → **replyboard-mapper.xml** 파일을 생성하고 ReplyBoardMapper.java에서 생성한 메소드와 연결이 필요한 SQL 쿼리문을 작성한다.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="replyBoard.mapper.ReplyBoardMapper">
  <!-- 테이블 추가 -->
  <!--
    ref=0; //글그룹 = 쿼리 실행시 가장 큰 ref값을 가져온 후 더하기 1을 한다.
    re_step=1; //새로운 글은 부모글이므로 1
    re_level=1;
  -->
  <insert id="insertReplyBoard" parameterType="rDTO">
    INSERT INTO replyBoard(writer,email,subject,password,ref,re_step,re_level,content)
    VALUES(#{writer},#{email},#{subject},#{password},
    <!-- 별칭 'r'을 주어 별도 쿼리로 인식시킴 -->
    (SELECT IFNULL(MAX(r.ref) + 1, 1) FROM replyBoard r),1,1,#{content})
  </insert>

  <!-- public List<ReplyBoardDTO> getAllReplyBoard(); -->
  <!-- 부모글 → 답글 → 답글의 답글 순서로 정렬한다. -->
  <select id="getAllReplyBoard" resultType="rDTO">
    SELECT * FROM replyBoard
    ORDER BY ref DESC, re_step ASC
  </select>
```



➤ 답글 게시판 작성에 필요한 replayboard-mapper.xml 파일 생성

- src/main/resources → templates → mybatis 폴더 → mappers 폴더 → replyboard-mapper.xml 파일을 생성하고 ReplyBoardMapper.java에서 생성한 메소드와 연결이 필요한 SQL 쿼리문을 작성한다.

```
<!-- public List<ReplyBoardDTO> getAllReplyBoard(); -->
<!-- 부모글 → 답글 → 답글의 답글 순서로 정렬한다. -->
<select id="getAllReplyBoard" resultType="rDTO">
    SELECT * FROM replyBoard
    ORDER BY ref DESC, re_step ASC
</select>

<!-- ReplyBoardDTO getOneBoard(int num) -->
<select id="getOneBoard" resultType="rDTO" parameterType="int">
    SELECT * FROM replyBoard WHERE num=#{num}
</select>

<!-- public void reWriteInsert(ReplyBoardDTO rdto); : 답글 작성하기 -->
<insert id="reWriteInsert" parameterType="rDTO">
    INSERT INTO replyBoard(writer,email,subject,password,ref,re_step,re_level,content)
    VALUES(#{writer},#{email},#{subject},#{password},#{ref},#{re_step}+1,#{re_level}+1,#{content})
</insert>

<!-- public void reSqlUpdate(ReplyBoardDTO rdto); -->
<update id="reSqlUpdate" parameterType="rDTO">
    UPDATE replyBoard
    SET re_step = re_step + 1
    WHERE ref = #{ref} AND re_step > #{re_step}
</update>

</mapper>
```

답글 게시판 필요한 Controller





➤ **답글 게시판 작성에 필요한 Controller 각 메소드들을 작성한다.**

- src/main/java → replyboard 폴더 → controller → ReplyBoardController.java 파일을 생성한 후 아래 메소드들을 작성한다.

```
package replyboard.controller;

import java.util.List;

@Controller
public class ReplyBoardController {

    @Autowired
    ReplyBoardService replyBoardService;

    // 게시물 목록으로 이동하는 컨트롤러
    @GetMapping("/board/list")
    public String boardList(Model model) {
        System.out.println("ReplyBoardController boardList() 호출");
        List<ReplyBoardDTO> replyList = replyBoardService.getAllReplyBoard();

        model.addAttribute("rlist", replyList);

        return "/replyBoard/replyboardList";
    }

    // 1. 글쓰기 폼으로 이동하는 컨트롤러
    @GetMapping("/board/writer")
    public String boardWriterForm() {
        System.out.println("ReplyBoardController boardWriterForm() 호출");
        String nextPage = "replyBoard/replyboardWrite_Form";
        return nextPage;
    }
}
```



➤ 답글 게시판 작성에 필요한 Controller 각 메소드들을 작성한다.

```
// 2. 글쓰기를 처리하는 컨트롤러
@GetMapping("/board/writerPro")
public String boardWriterPro(Model model, ReplyBoardDTO rdto) {
    System.out.println("ReplyBoardController boardWriterPro() 호출 함");
    replyBoardService.insertReplyBoard(rdto);
    return "redirect:/board/list";
}

// 3. 하나의 게시물 정보로 이동하는 컨트롤러
@GetMapping("/board/detail")
public String getOneBoard(@RequestParam("num") int num, Model model) {
    System.out.println("ReplyBoardController getOneBoard() 호출 함");

    ReplyBoardDTO oneList = replyBoardService.getOneBoard(num);
    model.addAttribute("onelist", oneList);

    return "/replyBoard/replyboardDetail";
}

// 4. 답글 작성하는 폼으로 이동하는 컨트롤러
@GetMapping("/board/reply")
public String reWriteForm(Model model, @RequestParam("num") int num,
    @RequestParam("ref") int ref,
    @RequestParam("re_step") int re_step,
    @RequestParam("re_level") int re_level
    ) {

    model.addAttribute("num", num);
    model.addAttribute("ref", ref);
    model.addAttribute("re_step", re_step);
    model.addAttribute("re_level", re_level);

    return "/replyBoard/replyboardReWrite_Form";
}

// 5. 답글 작성을 처리하는 컨트롤러
@PostMapping("/board/reWritePro")
public String reWritePro(ReplyBoardDTO rdto) {
    replyBoardService.replyProcess(rdto);
    return "redirect:/board/list";
}
```

게시판 필요한 각종 UI 화면





➤ 답글 게시판 – 전체 목록 보기 화면

- src/main/resources → templates → replyBoard 폴더 → replyboardList.html 파일생성 후 전체 게시글 목록 UI 화면을 작성한다.

전체 게시글 목록

전체 게시글 목록				
				글쓰기
번호	제목	작성자	작성일자	조회수
5	원글5	나리	2026-02-06 05:35	0
4	원글4	관리자	2026-02-06 05:35	0
3	원글3	길동	2026-02-06 05:35	0
2	원글2	달래	2026-02-06 05:35	0
1	원글1	오티	2026-02-06 05:35	0

[illegible]



➤ 답글 게시판 – 게시글 쓰기 화면

- src/main/resources → templates → replyBoard 폴더 → replyboardWrite_Form.html 파일생성

게시글 쓰기

작성자	
제목	
이메일	
비밀번호	
글내용	
글쓰기 다시작성 목록보기	



➤ 답글 게시판 – 게시글 상세 보기 화면

- src/main/resources → templates → replyBoard 폴더 → replyboardDetail.html 파일생성

게시글 보기

글번호	2	조회수	0
작성자	달래	작성일	2026-02-04 17:45
이메일	b@naver.com		
제목	답글 테스트02		
글내용	테스트02		
답글쓰기 수정하기 삭제하기 목록보기			



➤ 답글 게시판 – 게시물 상세 보기 화면

- src/main/resources → templates → replyBoard 폴더 → replyboardDetail.html 파일생성

```
<div class="container">
  <h2>게시글 보기</h2>
  <table width="600" border="1">
    <tr height="40">
      <td align="center" width="120">글번호</td>
      <td align="center" width="180" th:text="${onelist.num}"></td>
      <td align="center" width="120">조회수</td>
      <td align="center" width="180" th:text="${onelist.readcount}"></td>
    </tr>
    <tr height="40">
      <td align="center" width="120">작성자</td>
      <td align="center" width="180" th:text="${onelist.writer}"></td>
      <td align="center" width="120">작성일</td>
      <td align="center" width="180" th:text="${#dates.format(onelist.reg_date, 'yyyy-MM-dd HH:mm')}"></td>
    </tr>
    <tr height="40">
      <td align="center" width="120">이메일</td>
      <td align="center" colspan="3" th:text="${onelist.email}"></td>
    </tr>
    <tr height="40">
      <td align="center" width="120">제목</td>
      <td align="center" colspan="3" th:text="${onelist.subject}"></td>
    </tr>
    <tr height="80">
      <td align="center" width="120">글내용</td>
      <td align="center" colspan="3" th:text="${onelist.content}"></td>
    </tr>
    <tr height="40">
      <td align="center" colspan="4">
        <input type="button" value="답글쓰기"
          th:onclick="|location.href='@{/board/reply(
            num=${onelist.num},
            ref=${onelist.ref},
            re_step=${onelist.re_step},
            re_level=${onelist.re_level}
          )}'|">
        <input type="button" value="수정하기" th:onclick="|location.href='@{/board/list}'|">
        <input type="button" value="삭제하기" th:onclick="|location.href='@{/board/list}'|">
        <input type="button" th:onclick="|location.href='@{/board/list}'|" value="목록보기">
      </td>
    </tr>
  </table>
</div>
```



➤ 답글 게시판 – 답글 쓰기 화면

- src/main/resources → templates → replyBoard 폴더 → replyboardReWrite_Form.html 파일생성

답변글 작성하기

작성자	
제목	
이메일	
비밀번호	
글내용	
답글쓰기완료 다시작성 목록보기	



➤ 답글 게시판 - 답글 쓰기 화면

- src/main/resources → templates → replyBoard 폴더 → replyboardReWrite_Form.html 파일생성 후
- 답글을 작성하고 [답글쓰기완료] 버튼을 클릭하면 board/list로 이동한다.

전체 게시물 목록

					글쓰기
번호	제목	작성자	작성일자	조회수	
6	원글5	나리	2026-02-06 05:35	0	
5	↳[re] 댓글	관리자	2026-02-06 05:42	0	
4	원글4	관리자	2026-02-06 05:35	0	
3	원글3	길동	2026-02-06 05:35	0	
2	원글2	달래	2026-02-06 05:35	0	
1	원글1	오티	2026-02-06 05:35	0	

	num	writer	email	subject	password	reg_date	ref	re_step	re_level	readcount	content
▶	1	오티	oti@test.com	원글1	1234	2026-02-06 05:35:58	1	1	1	0	원글1 내용입니다
	2	달래	dalae@test.com	원글2	1234	2026-02-06 05:35:58	2	1	1	0	원글2 내용입니다
	3	길동	gildong@test.com	원글3	1234	2026-02-06 05:35:58	3	1	1	0	원글3 내용입니다
	4	관리자	admin@test.com	원글4	1234	2026-02-06 05:35:58	4	1	1	0	원글4 내용입니다
	5	나리	nari@test.com	원글5	1234	2026-02-06 05:35:58	5	1	1	0	원글5 내용입니다
	6	관리자	kjb1045@naver.com	댓글	1234	2026-02-06 05:42:31	5	2	2	0	댓글 답변